

TP2 - Testing práctico y contratos

Materia: Metodología de Sistemas II

Unidad 3: Verificación y Validación

Alumno: Nicolás Viruel

Laboratorio: módulo pedidos_descuento.py + test_pedidos_descuento.py (Python 3, unittest)

1. Patrón AAA en pruebas unitarias

Función bajo prueba: `calcular_descuento_envio(peso_kg, es_cliente_premium, cupon_activo)` - reglas de negocio de descuento sobre envío en un sistema de pedidos.

Criterio AAA: cada test tiene bloques comentados Arrange (datos de entrada), Act (una sola llamada al sistema) y Assert (comparación del resultado o excepción esperada).

Ejemplo representativo:

```
def test_no_premium_cupon_estandar(self) -> None:
    # Arrange
    peso = 10.0
    premium = False
    cupon = True

    # Act
    descuento = calcular_descuento_envio(peso, premium, cupon)

    # Assert
    self.assertEqual(descuento, 10.0)
```

La suite completa cubre premium con/sin cupón, peso alto, cliente estándar y validación de peso inválido. Ejecución:

```
python -m unittest test_pedidos_descuento.py -v
```

2. Dobles de prueba: Stub vs Mock (API externa de pagos)

Escenario: el servicio CheckoutService autoriza un pago llamando a PasarelaPagosExterna.cobrar(monto, token_tarjeta) vía HTTPS. En pruebas no se debe golpear la pasarela real ni depender de red.

Cuándo usar un Stub

Un Stub responde con datos preprogramados para alimentar indirectamente al componente bajo prueba. No verifica *cómo* fue invocado.

- Caso: probar que, si la pasarela devuelve { "estado": "aprobado", "id_transaccion": "TX-99" }, el pedido pasa a estado PAGADO y se persiste el id.
- Aislamiento: el stub sustituye la respuesta HTTP; el test no valida headers ni reintentos.
- Justificación: el foco es la transformación de datos posteriores al cobro, no el protocolo de la API.

Cuándo usar un Mock

Un Mock es un doble que registra y verifica interacciones (llamadas, orden, parámetros). Es indispensable cuando el contrato de integración es el riesgo.

- Caso: la pasarela exige enviar Idempotency-Key único por intento; el sistema debe reintentar solo ante timeout 504 y

nunca duplicar cobros ante 200.

- Mock: `mock_pasarela.cobrar.assert_called_once_with(monto=1500.0, token="tok_x", idempotency_key="uuid-1")` y simular secuencia 504 → 200.
- Justificación: se audita el protocolo de interacción; un stub no alcanza porque no falla si se omite la clave o se llama dos veces.

Resumen arquitectónico

Doble — Propósito — Pregunta que responde

Stub — Datos indirectos controlados — ¿Qué hace mi código con esta respuesta?

Mock — Contrato y comportamiento de colaborador — ¿Mi código invoca bien al externo?

En capas hexagonales, ambos se inyectan en el puerto `PasarelaPagos`; la diferencia está en si la aserción final mira estado de dominio (stub) o expectativas de llamada (mock).

3. Branch Coverage vs Statement Coverage

Segmento analizado

Función `calcular_descuento_envio` (ver `pedidos_descuento.py`): condicionales anidados (`premium + cupon`, `peso > 30`, ramas de cliente estándar).

Por qué Branch Coverage es más riguroso

- Statement coverage marca una línea como ejecutada si corrió al menos una vez; puede dejar sin probar combinaciones (p. ej. ejecutar solo `premium` sin cupón y dar por cubierta la línea del `if cupon_activo` falsa sin recorrer el camino `premium+cupón`).
- Branch coverage exige ejecutar cada salida de cada decisión (`true/false` de cada predicado). Detecta errores en ramas else olvidadas y en condiciones compuestas mal diseñadas.

Mapa de ramas y casos para 100% branch coverage

— Predicado — Resultado — Test que lo ejecuta

B1 — `peso_kg <= 0` — `true` — `test_peso_invalido_lanza_error`

B2 — `peso_kg <= 0` — `false` — cualquier test con `peso > 0`

B3 — `es_cliente_premium` — `true` — tests `premium`

B4 — `es_cliente_premium` — `false` — tests `no premium`

B5 — `cupon_activo` (dentro `premium`) — `true` — `test_premium_con_cupon_envio_gratis`

B6 — `cupon_activo` (dentro `premium`) — `false` — `test_premium_sin_cupon_descuento_fijo`

B7 — `peso_kg > 30` (no `premium`) — `true` — `test_no_premium_peso_alto`

B8 — `peso_kg > 30` (no `premium`) — `false` — tests con `peso 10`

B9 — `cupon_activo` (estándar) — `true` — `test_no_premium_cupon_estandar`

B10 — `cupon_activo` (estándar) — `false` — `test_no_premium_sin_cupon_tarifa_default`

Con los seis tests de la suite se cubren todas las ramas del segmento. Reporte típico (`coverage run -m unittest + coverage report --show-missing`): 100% branches en `pedidos_descuento.py` frente a ~85% statements si faltara alguna rama false.

Cierre

El patrón AAA mantiene tests legibles y auditables; stub y mock se eligen según si se valida datos o contrato con la pasarela; la cobertura de ramas fuerza el diseño de casos que statement coverage puede dar por suficientes sin serlo.